

Introduction to OOPS Programming

Q1. Compiler-Based Language (C) vs Interpreted Language

A compiler-based language like C is more suitable for a student productivity system in scenarios requiring speed, efficiency, and standalone deployment.

Execution: C compiles the entire source code into machine code before execution. The CPU directly runs binary instructions without any runtime translation, making operations like calculations and file I/O significantly faster than interpreted languages (e.g., Python) which translate code line-by-line at runtime.

Deployment: A compiled C program produces a standalone executable that runs on any compatible machine without requiring an interpreter or runtime environment — ideal for embedded or resource-constrained systems.

C also provides direct memory control via pointers, making it efficient for performance-critical tasks like storing and processing large student datasets.

Q2. Loops, Conditions, and Data Types for Weekly Performance

Data Types: Use float/double for study hours (to handle decimals like 5.5 hrs), int for day counters, and char arrays for remarks.

Loops: A for loop is ideal since the number of iterations is fixed at 7 days. It collects each day's hours and accumulates the total.

Conditions: if-else if-else chains compare the average against thresholds and display remarks such as 'Excellent' (avg ≥ 8), 'Good' (≥ 5), or 'Needs Improvement' (below 5).

This combination ensures structured data collection, accurate float-based calculation, and meaningful conditional feedback.

Q3. Why Loops are Essential in Continuously Running Programs

In a menu-driven system that runs until the user exits, loops are indispensable:

- A while(1) or do-while loop keeps the menu active and repeats after every user action without rewriting code.
- Repeating conditional statements instead would make the code exponentially long, unmanageable, and impossible to maintain.
- Loops preserve program state across iterations — after each action, the user is brought back to the main menu automatically.
- They are memory-efficient, reusing the same stack frame instead of duplicating code blocks.

Q4. Issue with int for Study Hours and Averages

Problem: When study hours are stored as int, decimal values are truncated (e.g., 7.5 becomes 7). During average calculation, if both total and divisor are int, C performs integer division — discarding the remainder. Example: $47 / 7$ gives 6 instead of 6.71.

Correct Approach: Declare study hours as float or double. Use float division for the average: `float avg = total / 7.0f;` — the `7.0f` suffix forces floating-point division, giving accurate results like 6.71.

Q5. Debugging File Crash While Reading Stored Data

- Check file open success: Verify `fopen()` does not return `NULL`. Use `perror()` to get the OS-level error reason (file missing, permission denied, etc.).
- Verify file mode: Ensure mode matches the operation — "r" for text reading, "rb" for binary. A mismatch causes garbled data or crashes.
- Check for EOF and read errors: Use `feof()` and `ferror()` inside the read loop to detect unexpected end-of-file or read errors.
- Match data types: If data was written with `fwrite()` in binary mode, read it back with `fread()` using identical data types and sizes.
- Use GDB debugger: Set breakpoints at file operations to identify the exact line causing the crash and inspect variable states.
- Ensure proper `fclose()`: An unclosed file from a previous run can leave corrupt or incomplete data, causing crashes on re-read.

Q6. Structures vs Multiple Arrays for Student Profiles

Logical Grouping: A struct bundles all related attributes (name, roll number, age, hours, grade) into one entity. Multiple arrays scatter these logically related fields across memory.

Data Integrity: With parallel arrays, maintaining correct index mapping across all arrays during insert/delete is error-prone. A struct keeps all attributes of a student together automatically.

Readability: `struct Student s; s.name, s.rollNo` is self-documenting. Parallel arrays like `name[i], rollNo[i]` require mental tracking of index relationships.

Scalability: Adding a new field (e.g., email) requires adding one struct member vs adding a new array and updating all related logic.

Function Passing: A struct can be passed as a single argument, keeping function signatures clean. Parallel arrays require passing multiple pointers.